



REPÚBLICA DE ANGOLA
MINISTÉRIO DAS TELECOMUNICAÇÕES, TECNOLOGIAS DE INFORMAÇÃO E COMUNICAÇÃO SOCIAL
MINISTÉRIO DA EDUCAÇÃO
INSTITUTO DE TELECOMUNICAÇÕES

Relatório da Disciplina de SI

Arsenal Gaming Hub

Grupo 6-(KERNEL) Turma M:

- Gabriel de Carvalho – 15592
- Frasncisco Gaspar – 15721
- Gelson José Manuel – 15680

Novembro de 20

Sumário

1 Introdução 3

1.1.1 Fase 1: Brainstorming e Definição Conceptual 3
1.1.2 Fase 2: Planeamento Estratégico e Arquitetural 3
1.1.3 Fase 3: Modularização e Especificação Funcional 4
1.1.4 Fase 4: Modelação de Dados (MER e DER) 4
1.1.5 Fase 5: Desenvolvimento do Diagrama Relacional 4
1.1.6 Fase 6: Gestão de Projeto e Distribuição de Tarefas 4
1.1.7 Fase 7: Autenticação, Segurança e Moderação 5
1.1.8 Fase 8: Desenvolvimento Ativo e Implementação 5
1.1.9 Fase 9: Análise, Auditoria e Testes 5
1.1.10 Fase 10: Marketing, Posicionamento e Redes Sociais 5

1.2 Objectivos 6

2 Caso de estudo 7

3 Solução Proposta 8

3.1 Requisitos Funcionais 8

3.1.1 Contas e autenticação 8
3.1.2 Amizades 9
3.1.3 Grupos, canais e papéis 9
3.1.4 Mensagens 9
3.1.5 Voz, vídeo e partilha de ecrã 10
3.1.6 Jogos e pacotes de jogo 10
3.1.7 Atividade, perfil e estatísticas 10
3.1.8 Competições 11
3.1.9 Agenda e mini-jogos 11
3.1.10 Notificações e moderação 11

3.2 Requisitos Não Funcionais 12

3.3 Modelo Entidade Relacionamento 13

3.4 Diagrama Entidade Relacionamento 15

4 Conclusão 21

1 Introdução

O presente relatório, elaborado no âmbito da disciplina de Sistemas de Informação (SI) documenta o ciclo de desenvolvimento do projeto "Arsenal Gaming Hub". Esta plataforma propõe-se a ser um espaço social desenhado de raiz para a comunidade gamer, transcendendo as funcionalidades genéricas de comunicação para oferecer uma camada de "modding social" moldada pelos próprios utilizadores.

Metodologia e Fases de Desenvolvimento

O desenvolvimento do Arsenal Gaming Hub segue uma metodologia estruturada em dez fases distintas, garantindo o alinhamento entre os requisitos funcionais únicos da plataforma e as boas práticas de engenharia de software. Abaixo detalha-se o escopo técnico e estratégico de cada etapa:

1.1.1 **Fase 1: Brainstorming e Definição Conceptual**

Consolidação da visão do produto com foco no diferenciador real: os pacotes de jogo comunitários. Nesta fase, foram mapeadas as funcionalidades do "Núcleo Social" (chat, voz, partilha de ecrã) versus as funcionalidades inovadoras (roletas, memes de áudio, estatísticas agregadas). Foi também definida a estratégia de nicho inicial, selecionando 1-2 jogos ou comunidades específicas para testar o conceito antes de escalar, mitigando o risco de concorrer diretamente com infraestruturas maduras.

1.1.2 Fase 2: Planeamento Estratégico e Arquitetural

Definição do roadmap técnico e validação do modelo de negócio (mercado de cosméticos comunitários e subscrição premium sem anúncios). Estabelecimento das restrições técnicas, incluindo a necessidade de baixa latência e a integração futura com APIs de jogos para Rich Presence automático. Planeamento da arquitetura modular para permitir que a plataforma evolua de um complemento (companion app) para um ecossistema completo.

1.1.3 Fase 3: Modularização e Especificação Funcional

Decomposição do sistema em módulos coesos. Destaque para a separação crítica entre o Módulo de Comunicação Base (voz/vídeo/chat) e o Módulo de Pacotes de Jogo (lógica de roletas, gestão de assets comunitários, placares). Definição dos requisitos para funcionalidades transversais como o quadro "Looking For Group", clipes automáticos e mini-jogos de espera.

1.1.4 Fase 4: Modelação de Dados (MER e DER)

Conceção do Modelo Entidade-Relacionamento para suportar a complexidade dos dados gerados pela comunidade. Estruturação de entidades chave como `Utilizador`, `Comunidade/Grupo`, `Jogo`, `PacoteDeJogo`, `AssetComunitário` e `Estatística`. Garantia de que o modelo suporta relações muitos-para-muitos entre jogos e pacotes, bem como a auditoria de conteúdo gerado por utilizadores.

1.1.5 Fase 5: Desenvolvimento do Diagrama Relacional

Tradução do MER lógico para o esquema físico de base de dados. Normalização das tabelas para otimizar consultas de estatísticas cross-game e pesquisa de comunidades públicas. Definição de índices estratégicos para funcionalidades de descoberta e matchmaking interno.

1.1.6 Fase 6: Gestão de Projeto e Distribuição de Tarefas

Implementação de metodologias ágeis através da criação de um quadro Trello. Separação clara de responsabilidades entre backend (API de pacotes, autenticação), frontend (interface de chamadas, renderização de overlays) e community/moderação. Priorização das tarefas ligadas ao diferencial do projeto (pacotes de jogo) face às funcionalidades commoditized.

1.1.7 Fase 7: Autenticação, Segurança e Moderação

Desenvolvimento de sistemas robustos de identidade e proteção. Além da autenticação padrão, esta fase contempla a implementação de regras claras e ferramentas técnicas para a moderação de conteúdo gerado pela comunidade (figurinhas, áudios), reconhecida no documento base como um ponto crítico de risco desde o início do projeto.

1.1.8 Fase 8: Desenvolvimento Ativo e Implementação

Execução das tarefas definidas com foco na entrega incremental. Desenvolvimento prioritário do motor de "Pacotes de Jogo" e da sua integração automática em chamadas/partilhas de ecrã. Implementação paralela do núcleo social, preferencialmente via integração com bibliotecas WebRTC existentes ou APIs de terceiros para reduzir a carga de infraestrutura própria nesta fase estudantil.

1.1.9 Fase 9: Análise, Auditoria e Testes

Validação técnica e funcional do sistema. Verificação da performance das funcionalidades em tempo real (voz, partilha de ecrã) e testes de usabilidade dos pacotes de jogo com o público-alvo. Auditoria de segurança aos assets carregados pela comunidade e revisão da conformidade com o modelo de dados definido.

1.1.10 Fase 10: Marketing, Posicionamento e Redes Sociais

Desenvolvimento da estratégia de comunicação focada na proposta de valor central: as ferramentas específicas por jogo moldadas pela comunidade. Criação de materiais promocionais que destaquem o Arsenal Gaming Hub não como "mais um chat", mas como a evolução da experiência social em gaming. Identificação de micro-influencers e comunidades dos jogos-piloto para adoção inicial.

1.2 Objectivos

Objetivo Geral: Criar um ecossistema social especializado que enriqueça a experiência de jogo em grupo através de ferramentas contextuais moldadas pela comunidade ("Pacotes de Jogo"), superando as limitações das plataformas genéricas ao integrar funcionalidades específicas de cada título diretamente na interação social.

Objetivos Específicos:

- **Potenciar a Interação Social Contextualizada**: Facilitar a comunicação e o entretenimento entre jogadores ao fornecer automaticamente kits de ferramentas adaptados ao jogo em curso (figurinhas, memes de áudio, roletas), transformando momentos genéricos de chat ou chamada em experiências temáticas e imersivas.

- **Fomentar a Criatividade e Participação Comunitária**: Capacitar os utilizadores a criar, partilhar e monetizar os seus próprios ativos e funcionalidades para jogos específicos, estabelecendo uma camada de "modding social" onde a própria comunidade define e evolui as ferramentas de interação disponíveis.

- **Otimizar a Logística e Coordenação de Grupos**: Simplificar a organização de sessões de jogo e a formação de equipas através de funcionalidades integradas como quadros "Looking For Group", agendamento automático de "noites de jogo", enquetes rápidas e torneios internos, reduzindo o atrito de coordenação externa.

- **Centralizar a Identidade e Progressão do Jogador**: Unificar estatísticas, conquistas e histórico de múltiplos jogos num único perfil cross-game, permitindo que os utilizadores acompanhem a sua evolução e a dos amigos de forma agregada, independentemente da plataforma ou título jogado Deve constar de todas as fases do projecto.

2 Caso de estudo

Após um grupo de amigos se descordenar ao tentar jogar juntos online, eles chegaram à conclusão de que precisam criar um sistema de informação que permita a gestão de uma plataforma social para gamers, incluindo utilizadores, grupos/comunidades, jogos, itens de pacotes criados pela comunidade e atividades realizadas. Na plataforma existem utilizadores, criadores de conteúdo, moderadores e administradores. Para cada utilizador é importante saber o username, email, dados de autenticação, perfil, estado da conta e identidades externas associadas. São realizadas atividades sociais e de gaming chamadas, partilhas de ecrã, eventos, torneios e sessões de jogo para as quais é necessário saber o grupo/canal onde ocorreram, o jogo associado, o utilizador responsável, os itens de pacote utilizados, a duração e as datas previstas e efetivas. Os itens de pacote, como figurinhas, sons, roletas e placares, só podem ser utilizados em associação com atividades ou contextos específicos da plataforma. Cada item pertence a um jogo ou é compatível com um conjunto definido de jogos, sendo importante guardar o seu tipo, referência, autor, estado de moderação, revisão ativa e, quando aplicável, custo/preço. Um utilizador pode criar ou pertencer a vários grupos, mas cada grupo tem apenas um proprietário. Um grupo pode ter vários canais, mensagens, eventos e atividades, mas cada canal pertence a um único grupo. O sistema deve ainda garantir moderação de conteúdo, denúncias e registos de auditoria.Deve constar de todas as fases do projecto.

3 Solução Proposta

A solução proposta para o caso de estudo é o desenvolvimento do Arsenal Gaming Hub, uma plataforma social modular orientada para comunidades gamer, cujo diferencial central são os Pacotes de Jogo criados pela comunidade figurinhas, áudios/memes, roletas, placares, enquetes e competições específicas por jogo. Em vez de competir diretamente com plataformas já consolidadas, a solução aposta numa abordagem faseada, podendo iniciar como uma companion app focada nos pacotes de jogo e na gestão de grupos, eventos, atividades e conteúdos comunitários, enquanto o núcleo social de chat, voz, vídeo e partilha de ecrã é tratado como base necessária ou integrado gradualmente. O sistema deve suportar utilizadores, perfis, amizades, grupos, canais, jogos, plataformas, itens de pacotes com revisões, atividades/sessões, utilização de itens, estatísticas, torneios, eventos, mini-jogos, moderação e auditoria, garantindo segurança, controlo de permissões e aprovação de conteúdos gerados pela comunidade..

3.1 Requisitos Funcionais

Os requisitos funcionais descrevem o que o Arsenal Gaming Hub deve fazer na primeira versão e estão organizados por módulo. Consideram-se quatro atores: o visitante (sem sessão iniciada), o utilizador (com conta), o membro de grupo (com papel de dono, admin ou membro) e o administrador da plataforma, responsável pela aprovação de jogos e conteúdos e pela moderação global.

3.1.1 Contas e autenticação

RF01 O sistema deve permitir o registo e o início de sessão com email e palavra-passe.

RF02 O sistema deve permitir o registo e o início de sessão com contas externas (Google, Steam e Discord), podendo uma conta ter várias identidades associadas.

RF03 O sistema deve permitir a verificação do email e a recuperação da palavra-passe.

RF04 O utilizador deve poder editar o seu perfil (nome de exibição, avatar e biografia).

RF05 O utilizador deve poder listar as suas sessões ativas e terminá-las remotamente.

RF06 O utilizador deve poder eliminar a sua conta, sendo o seu conteúdo anonimizado.

3.1.2 Amizades

RF07 O utilizador deve poder enviar, aceitar, recusar e cancelar pedidos de amizade.

RF08 O utilizador deve poder remover amigos e bloquear outros utilizadores; um utilizador bloqueado não pode enviar pedidos de amizade nem mensagens diretas.

RF09 O utilizador deve poder consultar o estado e o jogo atual dos seus amigos.

3.1.3 Grupos, canais e papéis

RF10 O utilizador deve poder criar um grupo privado, tornando-se o dono desse grupo.

RF11 Os admins do grupo devem poder criar, renomear, reordenar e eliminar canais de texto e de voz.

RF12 O sistema deve permitir gerar convites com código, data de validade e limite de utilizações; a entrada num grupo só é possível através de um convite válido.

RF13 O sistema deve aplicar papéis fixos dentro do grupo: o dono tem todas as permissões (incluindo eliminar o grupo e transferir a propriedade); o admin gere canais e convites, remove membros, apaga mensagens e cria torneios e eventos; o membro utiliza os canais e participa nas atividades.

RF14 O membro deve poder sair do grupo e os admins podem expulsar membros; o dono tem de transferir a propriedade antes de sair.

RF15 O sistema deve registar em auditoria as ações administrativas realizadas no grupo.

3.1.4 Mensagens

RF16 O utilizador deve poder enviar mensagens de texto em canais e em conversas diretas.

RF17 O utilizador deve poder responder a mensagens e criar threads.

RF18 O utilizador deve poder reagir às mensagens com emojis.

RF19 O utilizador deve poder enviar anexos (imagens, vídeos e ficheiros), respeitando os limites de tamanho e os tipos permitidos.

RF20 O utilizador deve poder editar e apagar as suas mensagens, mantendo-se o histórico de edições; os admins podem apagar mensagens de outros membros.

RF21 O sistema deve carregar o histórico de um canal com paginação.

3.1.5 Voz, vídeo e partilha de ecrã

RF22 O utilizador deve poder entrar e sair de canais de voz persistentes e ver quem está ligado.

RF23 O utilizador deve poder ligar e desligar o microfone e a câmara e partilhar o ecrã.

RF24 O sistema deve registar as sessões de voz (entrada, saída e utilização de vídeo ou de partilha de ecrã).

RF25 O sistema deve disponibilizar uma soundboard para reproduzir sons do jogo durante a chamada, de forma que todos os participantes os ouçam.

RF26 Os admins devem poder configurar o número máximo de participantes por canal de voz.

3.1.6 Jogos e pacotes de jogo

RF27 O utilizador deve poder pesquisar e listar os jogos aprovados e as respetivas plataformas.

RF28 O utilizador deve poder submeter novos jogos, que ficam pendentes até à aprovação de um administrador da plataforma.

RF29 Cada jogo tem um único pacote; qualquer utilizador pode propor itens (figurinhas, sons e roletas) e edições a esses itens.

RF30 Cada alteração a um item deve gerar uma revisão pendente, sendo visível apenas a revisão aprovada.

RF31 O administrador da plataforma deve poder aprovar ou rejeitar jogos e revisões (indicando o motivo) e reverter um item para uma revisão anterior.

RF32 O utilizador deve poder usar as figurinhas do jogo nas conversas.

RF33 O utilizador deve poder criar roletas configuráveis (por exemplo, sorteio de mapa, personagem ou desafio) e girá-las num canal, com o resultado visível para todos.

RF34 O utilizador deve poder criar enquetes rápidas num canal, com um voto por utilizador e encerramento manual ou automático.

RF35 Quando o utilizador define o jogo e entra numa chamada ou partilha o ecrã, o sistema deve sugerir automaticamente o kit de ferramentas desse jogo.

3.1.7 Atividade, perfil e estatísticas

RF36 O sistema deve registar as sessões de jogo (jogo, plataforma, início e fim), de forma automática quando existir cliente desktop e manual nos restantes casos.

RF37 O sistema deve mostrar no perfil as horas jogadas por jogo, calculadas a partir das sessões.

RF38 Cada jogo deve definir os tipos de estatística (por exemplo, K/D e troféus) e o utilizador preenche os respetivos valores manualmente.

RF39 O sistema deve mostrar o indicador de plataforma (PC, PS5, Xbox, Switch ou mobile).

3.1.8 Competições

RF40 Os admins devem poder criar torneios num grupo, escolhendo o jogo, o formato (eliminatória, liga ou fase de grupos) e a configuração.

RF41 O sistema deve permitir inscrever jogadores e equipas; um jogador individual é tratado como uma equipa de um membro.

RF42 O sistema deve gerar as fases e as partidas de acordo com o formato escolhido.

RF43 O sistema deve registar os resultados das partidas e atualizar a classificação.

RF44 O utilizador deve poder consultar os placares de cada torneio.

3.1.9 Agenda e mini-jogos

RF45 Os admins devem poder criar eventos de noites de jogo (jogo, canal, data e hora).

RF46 O membro deve poder confirmar a presença nos eventos (vou, talvez ou não vou).

RF47 O sistema deve enviar lembretes automáticos aos participantes antes do início do evento, com antecedência configurável.

RF48 O membro deve poder iniciar sessões de mini-jogos num canal, com entrada de participantes, pontuação e ranking.

3.1.10 Notificações e moderação

RF49 O sistema deve apresentar notificações na aplicação (pedidos de amizade, convites, lembretes e respostas).

RF50 O utilizador deve poder denunciar mensagens, utilizadores e itens de pacote.

RF51 O administrador da plataforma deve poder analisar as denúncias e apagar conteúdo, suspender contas ou expulsar utilizadores.

3.2 Requisitos Não Funcionais

Os requisitos não funcionais definem as qualidades que o sistema deve apresentar, como desempenho, segurança, privacidade e usabilidade. A Tabela 1 apresenta-os, com as metas propostas para a primeira fase, que servem de ponto de partida e podem ser ajustadas conforme a infraestrutura disponível.

Tabela 1 – Requisitos não funcionais do Arsenal Gaming Hub

ID	Categoria	Requisito	Meta proposta
RNF01	Desempenho	Entrega de mensagens em tempo real	p95 inferior a 300 ms
RNF02	Desempenho	Latência de voz e vídeo ponta a ponta	Voz inferior a 150 ms; vídeo inferior a 300 ms
RNF03	Desempenho	Tempo de resposta da API	p95 inferior a 300 ms
RNF04	Desempenho	Carregamento inicial da aplicação	Inferior a 3 s em rede 4G
RNF05	Escalabilidade	Crescimento horizontal, com a voz num serviço próprio (SFU/WebRTC)	10 mil contas e 1 000 utilizadores em simultâneo na primeira fase
RNF06	Disponibilidade	Serviço disponível	99,5% por mês
RNF07	Recuperação	Cópias de segurança diárias	Perda máxima de 24 h; restauro em até 4 h
RNF08	Fiabilidade	Reconexão automática do chat e da voz; confirmação de receção de cada mensagem	Nenhuma mensagem enviada com sucesso é perdida
RNF09	Segurança	Palavras-passe guardadas com argon2id ou bcrypt	Sem armazenamento em claro
RNF10	Segurança	Comunicações cifradas	TLS 1.2 ou superior; DTLS-SRTP na voz e no vídeo
RNF11	Segurança	Tokens de acesso de curta duração, com refresh rotativo	Expiração e revogação por sessão
RNF12	Segurança	Autorização por papel verificada em todos os pedidos	Sem confiar no cliente
RNF13	Segurança	Limitação de pedidos e validação dos ficheiros carregados	Proteção contra abuso e ficheiros maliciosos
RNF14	Privacidade	Conformidade com a legislação de proteção de dados pessoais (Lei n.º 22/11 de Angola, com o RGPD como referência)	Exportar e eliminar dados a pedido
RNF15	Privacidade	Deteção automática de jogos mediante consentimento	Desativada por omissão
RNF16	Privacidade	Mensagens diretas visíveis apenas aos participantes	Sem acesso por terceiros
RNF17	Integridade de dados	Integridade referencial, transações nas operações multi-tabela e apagamento lógico	Sem registos órfãos
RNF18	Ficheiros	Limites de tamanho por tipo de ficheiro	Anexo até 25 MB; som até 10 s e 500 KB; figurinha até 512 KB

ID	Categoria	Requisito	Meta proposta
RNF19	Usabilidade	Interface em português e inglês, adaptada a telemóvel	Internacionalização desde o início
RNF20	Acessibilidade	Contraste, navegação por teclado e leitores de ecrã	WCAG 2.1, nível AA
RNF21	Eficiência de dados	Débito adaptativo e modo de baixo consumo na voz e no vídeo	Funciona em redes lentas
RNF22	Compatibilidade	Navegadores recentes e dispositivos móveis	Cliente desktop opcional para deteção de jogos
RNF23	Manutenibilidade	Arquitetura modular, integração e entrega contínuas, migrações de BD versionadas e API documentada (OpenAPI)	Testes automatizados nas regras de negócio
RNF24	Observabilidade	Logs estruturados, métricas e alertas	Logs técnicos fora da base de dados principal
RNF25	Moderação	Tempo de revisão de conteúdo pendente	Até 48 h

3.3 Modelo Entidade Relacionamento

users(id; username; email; password_hash; is_platform_admin; status; created_at; deleted_at)

user_profiles(user_id; display_name; avatar_url; bio)

auth_identities(id; user_id; provider; provider_user_id)

sessions(id; user_id; device_info; refresh_token_hash; expires_at; revoked_at)

friendships(id; requester_id; addressee_id; status; created_at)

notifications(id; user_id; type; payload; read_at; created_at)

groups(id; name; icon_url; owner_id; created_at)

group_members(group_id; user_id; role; joined_at)

group_invites(id; group_id; code; created_by; expires_at; max_uses; use_count)

channels(id; group_id; name; type; position; created_at)

channel_participants(channel_id; user_id)

messages(id; channel_id; author_id; content; reply_to_id; thread_root_id; created_at; edited_at; deleted_at)

message_edits(id; message_id; previous_content; edited_at)

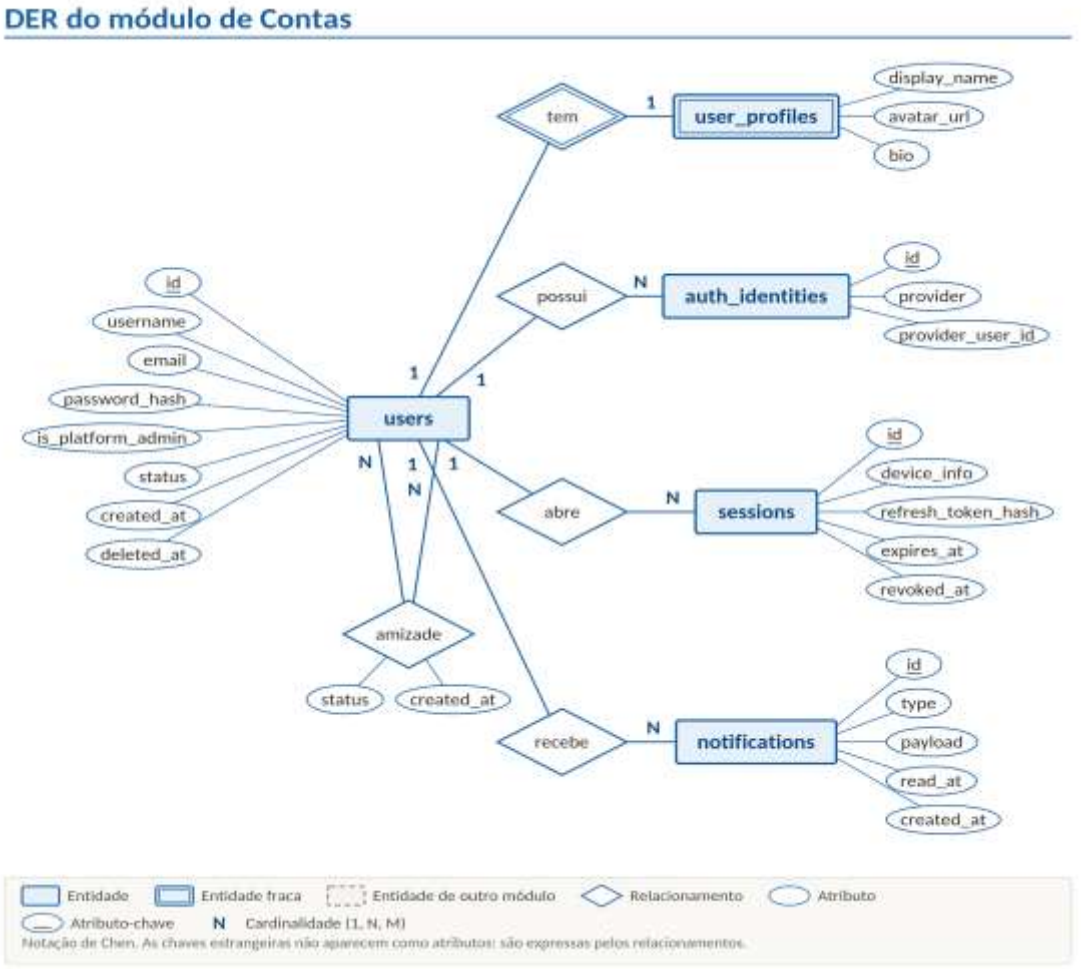
message_attachments(id; message_id; file_url; file_name; mime_type; size_bytes)

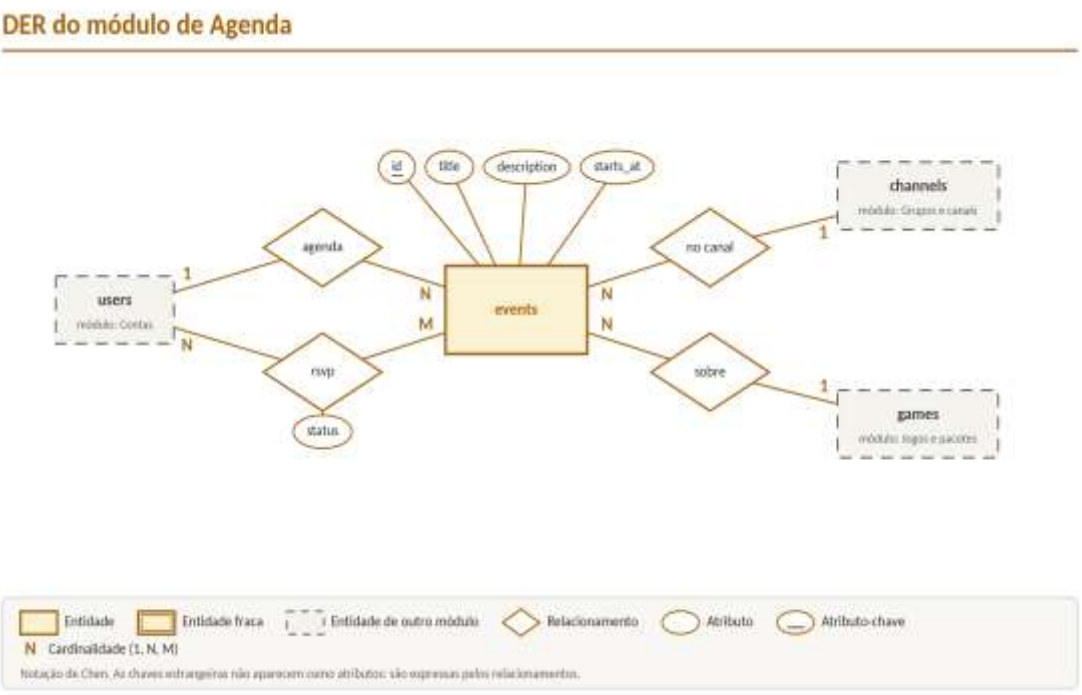
message_reactions(message_id; user_id; emoji; created_at)

voice_sessions(id; channel_id; user_id; started_at; ended_at; has_video; has_screenshare)

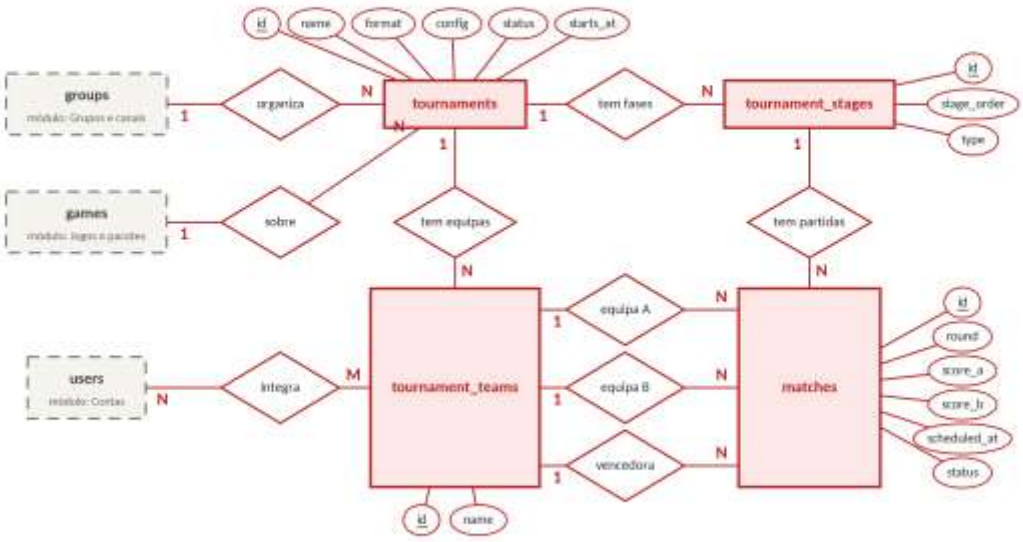
platforms(id; name)
games(id; name; cover_url; description; status; submitted_by; reviewed_by; reviewed_at)
game_platforms(game_id; platform_id)
pack_items(id; game_id; type; name; status; created_by; reviewed_by; reviewed_at; current_revision_id)
pack_item_revisions(id; item_id; version; file_url; data; edited_by; created_at)
polls(id; channel_id; game_id; created_by; question; closes_at)
poll_options(id; poll_id; text)
poll_votes(poll_id; user_id; option_id)
game_sessions(id; user_id; game_id; platform_id; started_at; ended_at)
game_stat_definitions(id; game_id; name; data_type; status)
user_game_stats(user_id; stat_definition_id; value; updated_at)
tournaments(id; group_id; game_id; name; format; config; status; starts_at; created_by)
tournament_stages(id; tournament_id; stage_order; type)
tournament_teams(id; tournament_id; name)
team_members(team_id; user_id)
matches(id; stage_id; round; team_a_id; team_b_id; score_a; score_b; winner_team_id; scheduled_at; status)
events(id; group_id; game_id; channel_id; title; description; starts_at; created_by)
event_rsvps(event_id; user_id; status)
minigames(id; name; slug; description)
minigame_sessions(id; minigame_id; channel_id; started_by; started_at; ended_at)
minigame_participants(session_id; user_id; score; rank)
reports(id; reporter_id; target_type; target_id; reason; status; handled_by; created_at)
audit_logs(id; group_id; actor_id; action; target_type; target_id; metadata; created_at)

3.4 Diagrama Entidade Relacionamento

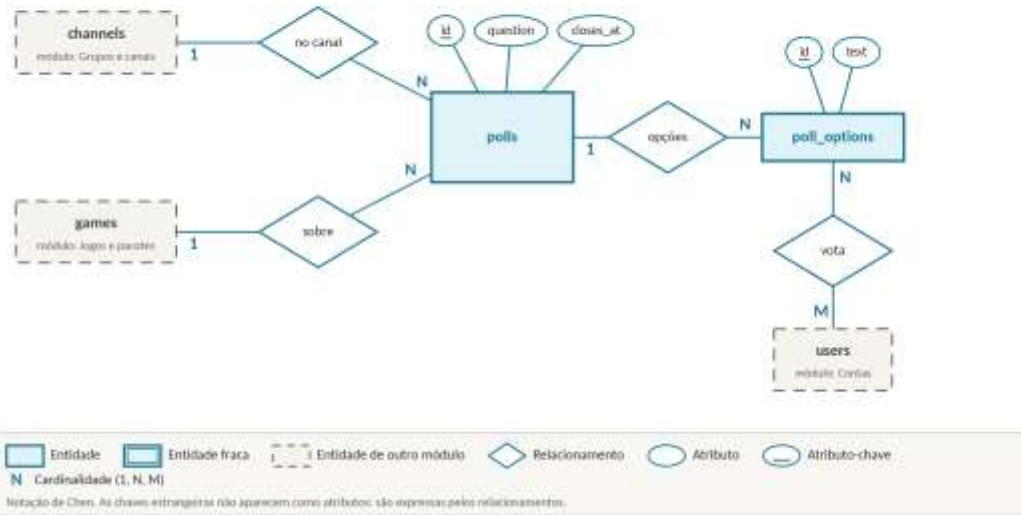




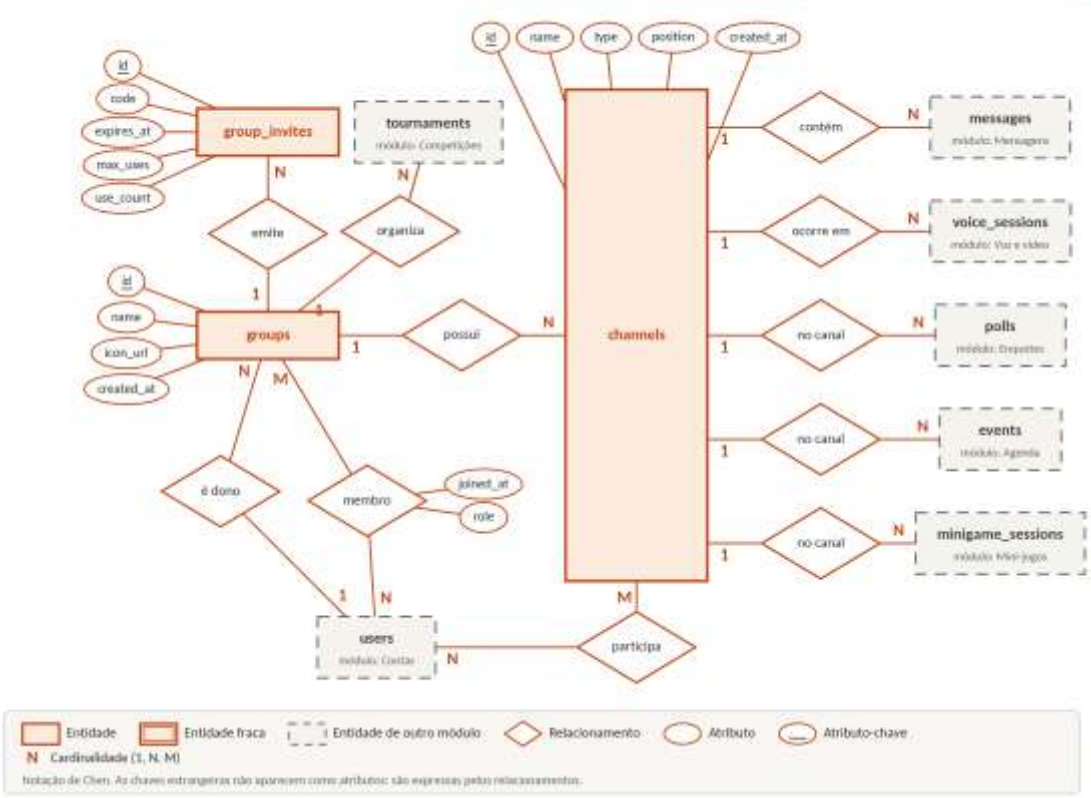
DER do módulo de Competições



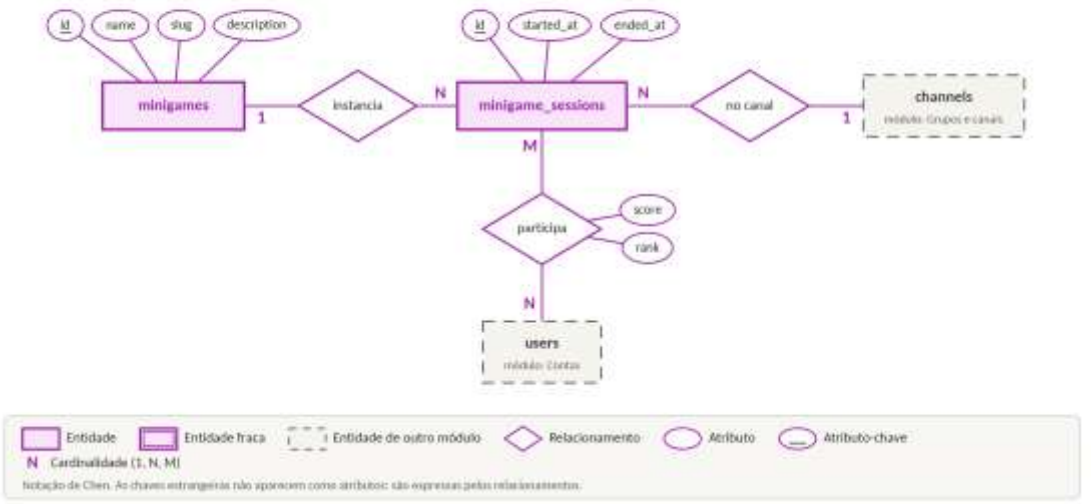
DER do módulo de Enquetes



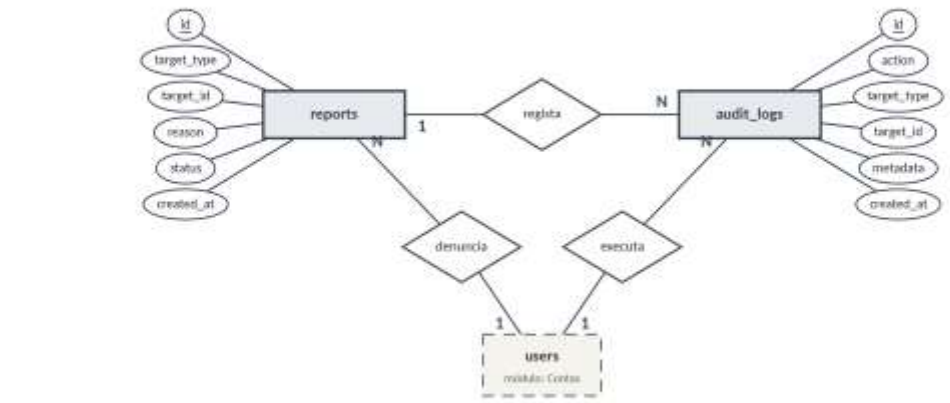
DER do módulo de Grupos e canais



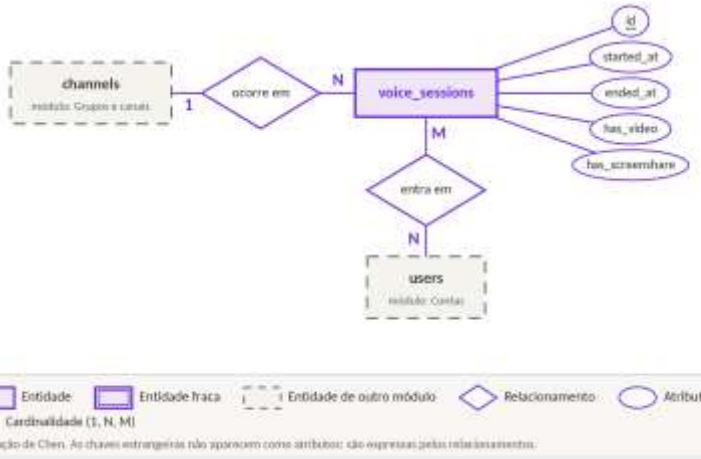
DER do módulo de Mini-jogos



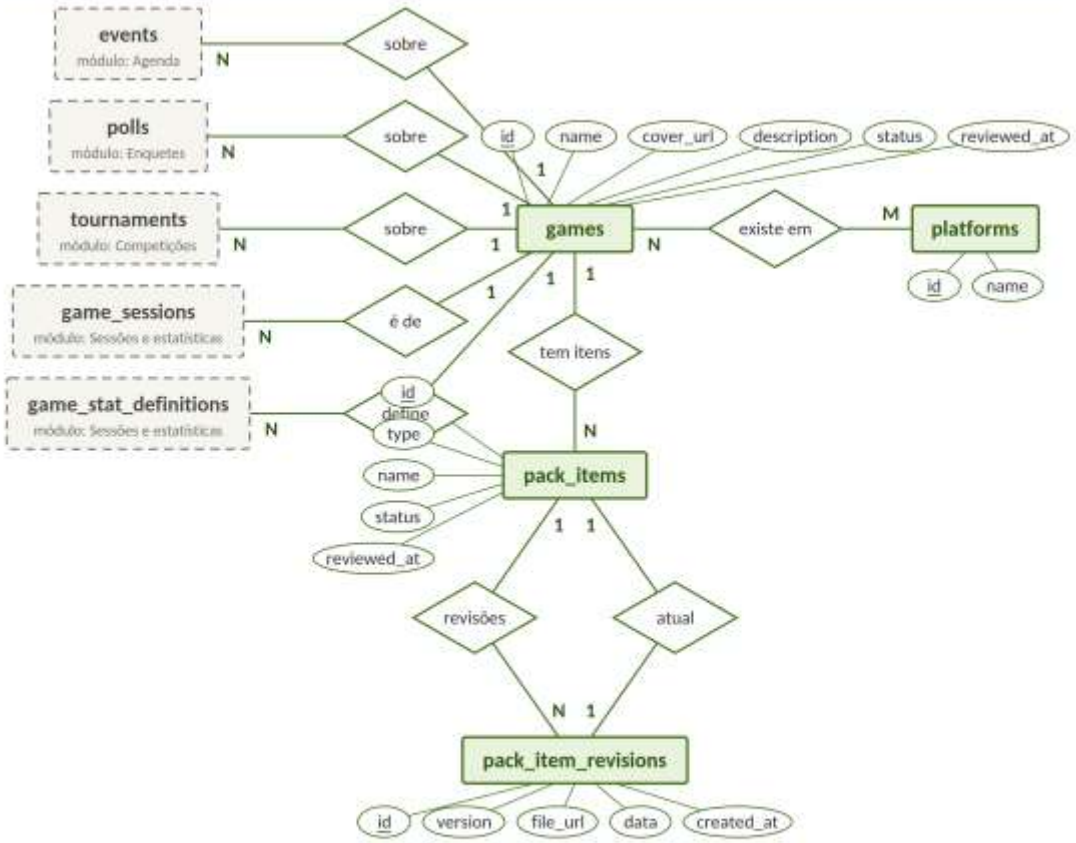
DER do módulo de Moderação



DER do módulo de Voz e vídeo



DER do módulo de Jogos e pacotes

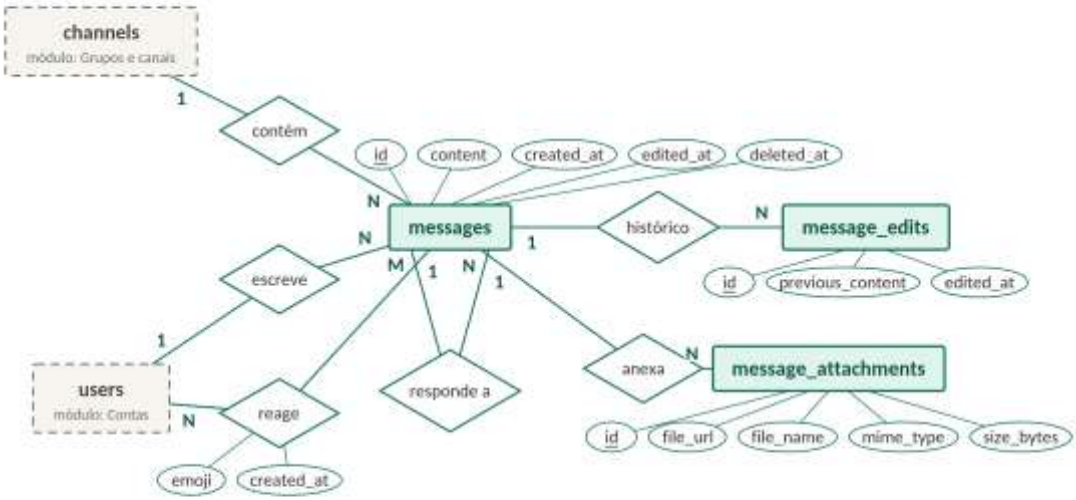


Entidade · Entidade fraca · Entidade de outro módulo · Relacionamento · Atributo · Atributo-chave

N Cardinalidade (1, N, M)

Notação de Chen. As chaves estrangeiras não aparecem como atributos: são expressas pelos relacionamentos.

DER do módulo de Mensagens

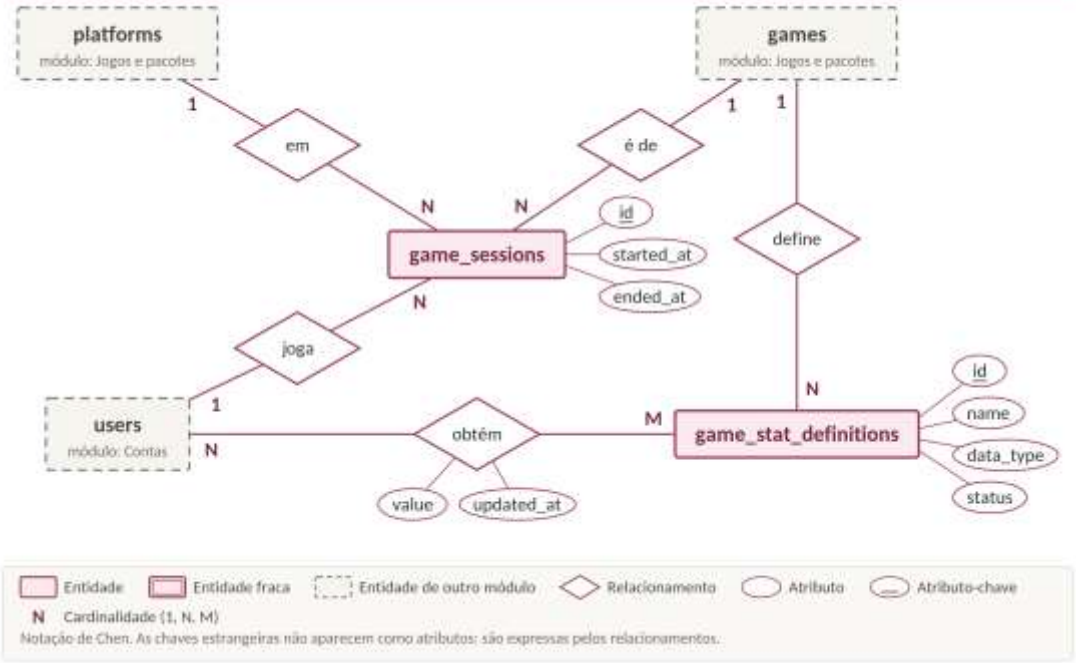


Entidade · Entidade fraca · Entidade de outro módulo · Relacionamento · Atributo · Atributo-chave

N Cardinalidade (1, N, M)

Notação de Chen. As chaves estrangeiras não aparecem como atributos: são expressas pelos relacionamentos.

DER do módulo de Sessões e estatísticas



4 Conclusão

O relatório analisou e modelou o Arsenal Gaming Hub, uma plataforma social para gamers centrada em pacotes de jogo criados pela comunidade, com 51 requisitos funcionais, 25 não funcionais e um modelo de dados de 39 tabelas.

Conclusões principais:

- Pacotes de jogo: são o diferencial. Cada jogo tem um pacote único, com itens, revisões e aprovação por um administrador, o que permite reverter alterações indevidas.
- Núcleo social (chat, voz, vídeo, partilha de ecrã): é necessário, mas não é a inovação, por isso deve usar tecnologias existentes como o WebRTC.
- Moderação, segurança, privacidade e desempenho em tempo real: foram pensados desde o início.

Limitações: a plataforma exige muito tempo, equipa e infraestrutura, e a concorrência é forte. As estatísticas manuais podem ser falsificadas, e a aprovação de todo o conteúdo pode tornar-se um estrangulamento. Recomenda-se começar com um ou dois jogos ou comunidades-piloto.

Trabalho futuro: monetização, clipes automáticos, Looking For Group, descoberta de comunidades, moderação partilhada e importação de estatísticas por APIs. O modelo já está preparado para isto.

Conclusão final: o projeto é viável se for faseado, e a sua proposta (ferramentas por jogo moldadas pela comunidade) responde a uma necessidade que as plataformas genéricas não cobrem.